M92U Image Transport集成
1. 代码提交
2. 包依赖关系图
3. 优化内容
3.1 发布端优化
3.2 订阅端优化
3.3 延时优化
4. 测试数据
4.1 PC上的测试数据
4.1.1 Realsense D455
2025年4月1日-优化了订阅端
4.1.2 肇观相机
2025年4月1日-优化了订阅端
4.1.3 多个订阅者
4.2 真机上的测试数据
4.2.1 Realsense D435
4.2.2 肇观相机
4.2.3 多个订阅者

# M92U Image Transport集成

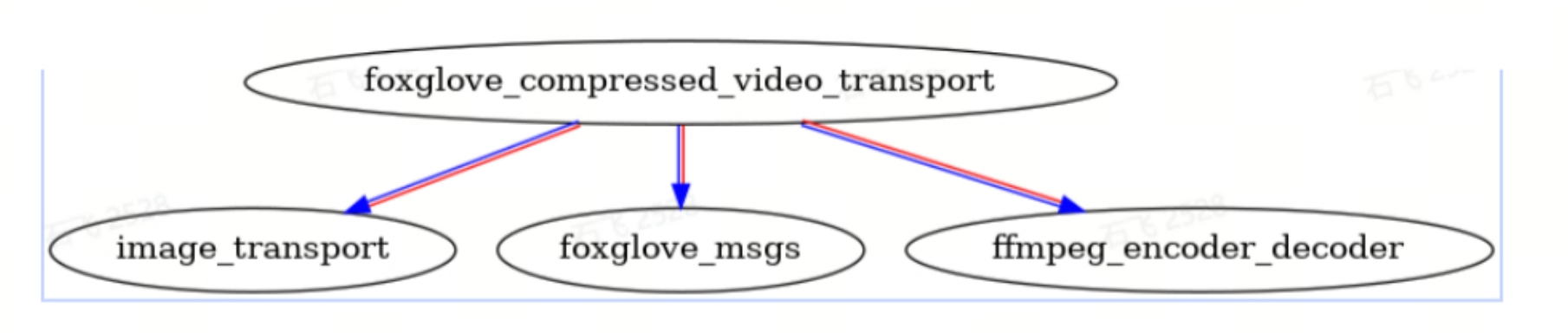
李向远 | 3月27日创建

| 版本 | 日期 | 内容 | 修订人 |
|---|---|---|---|
| v0.1 | 2025 年 03 月 27 日 | 初始版本 | 李向远 |
| v0.2 | 2025 年 04 月 1 日 | 优化订阅端，并补充真机测试数据 | 李向远 |
| | | | |

## 1. 代码提交

| | |
|---|---|
| 源码地址 | mirobotics/common/thirdparty |
| Deb地址 | mirobotics/common/debs/ceres/mirobot_thirdparty_deb |
| Deb包名 | mirobot-common-thirdparty_1.0.1_amd64.deb |

## 2. 包依赖关系图


foxglove_compressed_video_transport
image_transport
foxglove_msgs
ffmpeg_encoder_decoder

| Package Name | GitHub地址 | 分支 |
|---|---|---|
| foxglove_compressed_video_transport | foxglove_compressed_video_transport | 1.0.1 |
| foxglove_msgs | foxglove-sdk | sdk/v0.4.1 |
| ffmpeg_encoder_decoder | ffmpeg_encoder_decoder | 2.0.0 |
| image_transport | image_transport | humble |

## 3. 优化内容

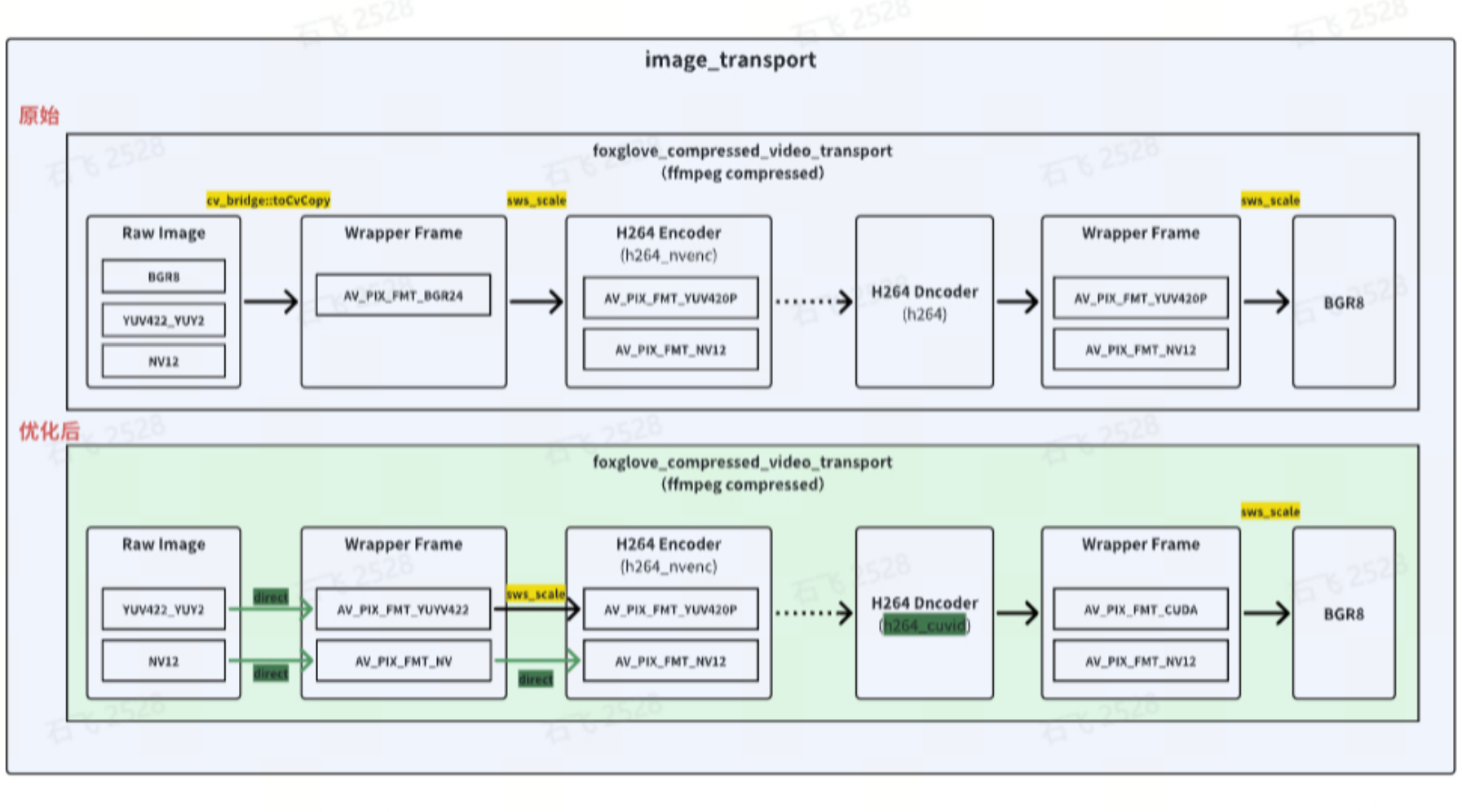
### 3.1 发布端优化

如下图所示，基于FFMPEG进行H.264编码，涉及两个格式：

- 输出Raw Data的格式（相机硬件输出）
- 编码器支持的Pixel Format

在很多情况下这两种格式是不同的，所以就引入了一个Wrapper Frame来做适配。原来的实现是通过 `cv_bridge` 将输入数据转为BGR8，然后再把它转成编码器所支持的格式。

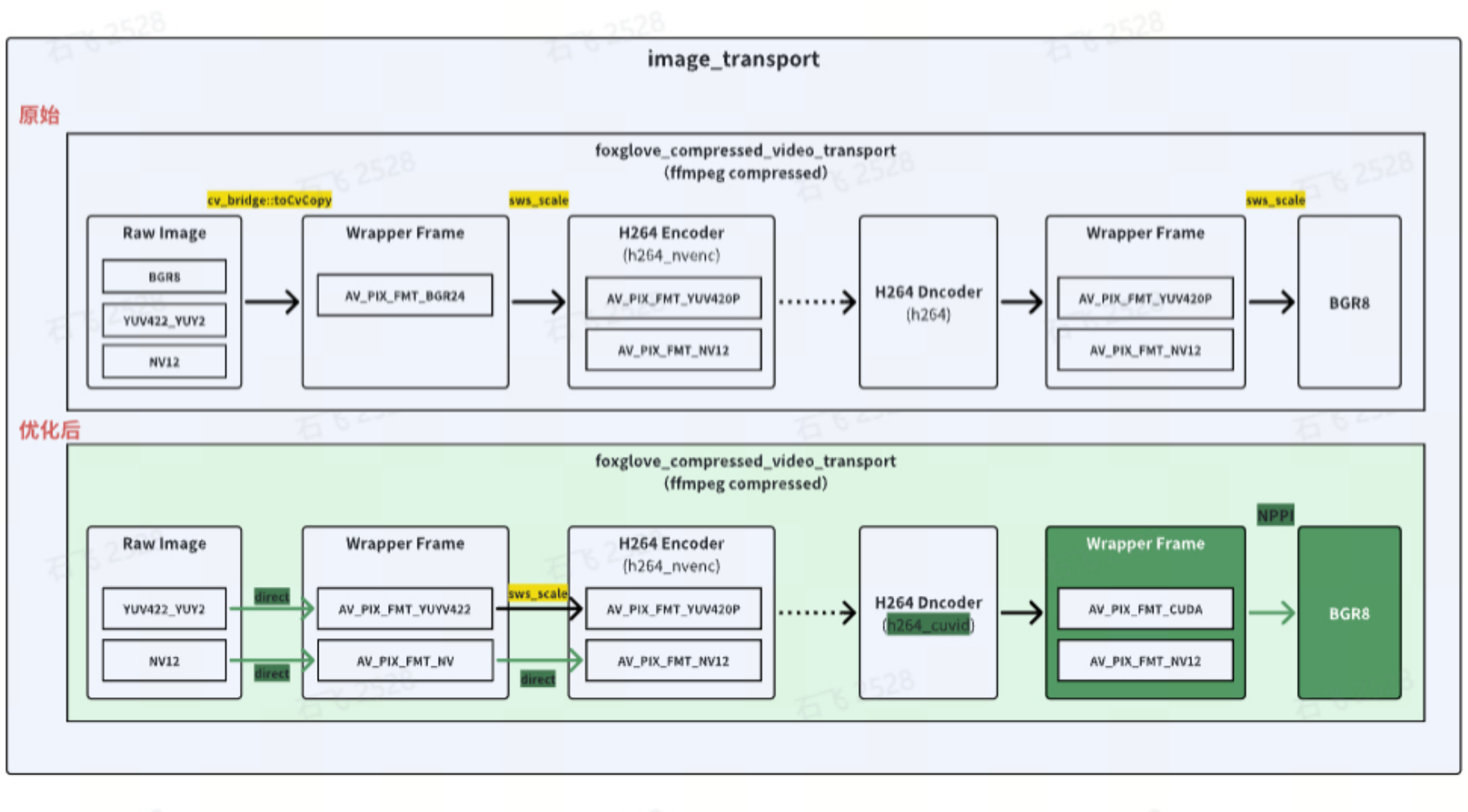
优化后，去掉了 `cv_bridge` 的转换环节，如果输入格式与编码器支持的格式一致（例如NV12），则去掉FFMPEG中的格式转换环节。


image_transport
原始
foxglove_compressed_video_transport
(ffmpeg compressed)
cv_bridge::toCvCopy
sws_scale
Raw Image
BGR8
YUV422_YUY2
NV12
Wrapper Frame
AV_PIX_FMT_BGR24
H264 Encoder
(h264_nvenc)
AV_PIX_FMT_YUV420P
AV_PIX_FMT_NV12
H264 Dncoder
(h264)
Wrapper Frame
AV_PIX_FMT_YUV420P
AV_PIX_FMT_NV12
sws_scale
BGR8
优化后
foxglove_compressed_video_transport
(ffmpeg compressed)
Raw Image
YUV422_YUY2
NV12
direct
direct
Wrapper Frame
AV_PIX_FMT_YUYV422
AV_PIX_FMT_NV
sws_scale
direct
H264 Encoder
(h264_nvenc)
AV_PIX_FMT_YUV420P
AV_PIX_FMT_NV12
H264 Dncoder
(h264_cuvid)
Wrapper Frame
AV_PIX_FMT_CUDA
AV_PIX_FMT_NV12
sws_scale
BGR8

> 由于ROS 2 Humble对NV12的支持有限：
>
> > Enhance NV12 and NV21 Support in sensor_msgs::image_encodings #264
> >
> > Add support for NV12 encoding in cv_bridge #544
>
> 所以当输出Raw Image是NV12格式时，无法准确描述，用的是 `sensor_msgs::image_encodings::MONO8`，造成的影响是用工具查看Raw Image时图象不正常。

### 3.2 订阅端优化

- 使用Nvidia的解码器 h264_cuvid
- 基于NVIDIA Performance Primitives (NPP)优化FFMPEG 的中 `sws_scale`。


image_transport
原始
foxglove_compressed_video_transport
(ffmpeg compressed)
cv_bridge::toCvCopy
sws_scale
Raw Image
BGR8
YUV422_YUY2
NV12
Wrapper Frame
AV_PIX_FMT_BGR24
H264 Encoder
(h264_nvenc)
AV_PIX_FMT_YUV420P
AV_PIX_FMT_NV12
H264 Dncoder
(h264)
Wrapper Frame
AV_PIX_FMT_YUV420P
AV_PIX_FMT_NV12
sws_scale
BGR8
优化后
foxglove_compressed_video_transport
(ffmpeg compressed)
Raw Image
YUV422_YUY2
NV12
direct
direct
Wrapper Frame
AV_PIX_FMT_YUYV422
AV_PIX_FMT_NV
sws_scale
direct
H264 Encoder
(h264_nvenc)
AV_PIX_FMT_YUV420P
AV_PIX_FMT_NV12
H264 Dncoder
(h264_cuvid)
NPPI
Wrapper Frame
AV_PIX_FMT_CUDA
AV_PIX_FMT_NV12
BGR8

### 3.3 延时优化

当用 `h264_cuvid` 解码时，CPU占用率要低于用 `h264` 解码器，但延时明显变大.

FFmpeg/libavcodec/cuviddec.c at master · FFmpeg/FFmpeg

```
ctx->cuparseinfo.ulMaxNumDecodeSurfaces = 1;
ctx->cuparseinfo.ulMaxDisplayDelay = (avctx->flags & AV_CODEC_FLAG_LOW_DELAY) ? 0 : CUVID
ctx->cuparseinfo.pUserData = avctx;
ctx->cuparseinfo.pfnSequenceCallback = cuvid_handle_video_sequence;
ctx->cuparseinfo.pfnDecodePicture = cuvid_handle_picture_decode;
ctx->cuparseinfo.pfnDisplayPicture = cuvid_handle_picture_display;
```

原因是默认缓冲了4帧，可以通过配置AV_CODEC_FLAG_LOW_DELAY来解决。

> 这里要注意FFMPEG的版本，该配置是在4.4版本中添加的，当前ubuntu 20.04默认FFMPEG版本是4.2.7，4帧延时是固定的，需要打patch。

## 4. 测试数据

- 用户 CPU 时间（`std::clock()`）与实际经过的时间（`std::chrono::steady_clock::now()`）之比。
- 计算时考虑了硬件线程数（CPU 核心数），因此得出的结果是相对于整个 CPU 的使用率。

公式上，CPU 使用率是通过如下公式计算的：

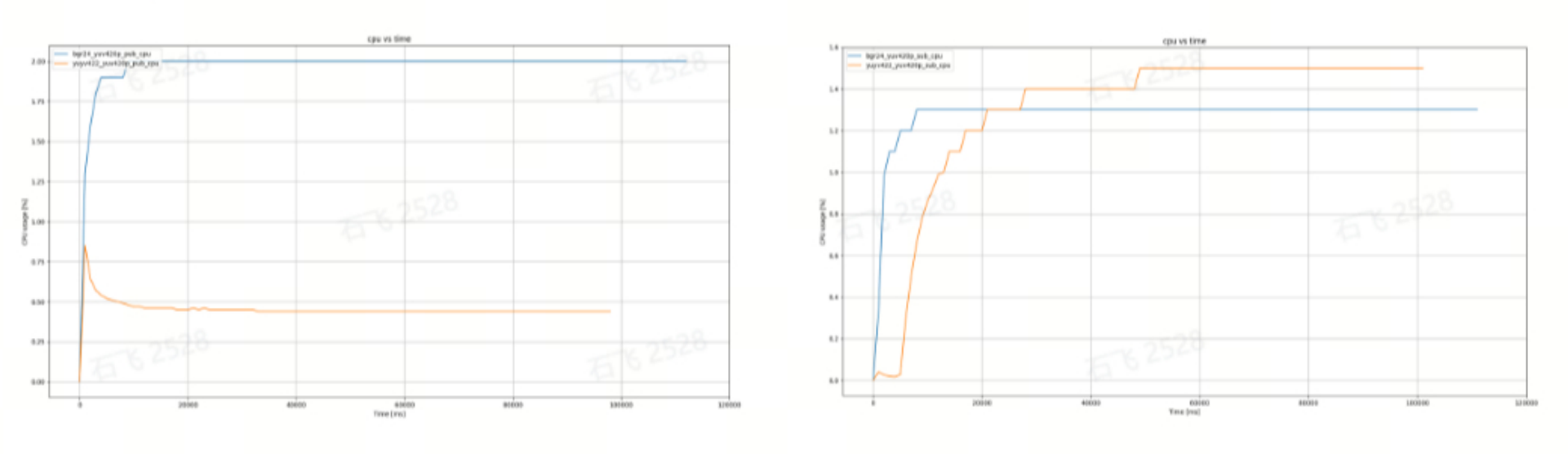
$$\text{CPU 使用率} = \frac{\text{用户 CPU 时间}}{\text{真实时间} \times \text{线程数}} \times 100$$

### 4.1 PC上的测试数据

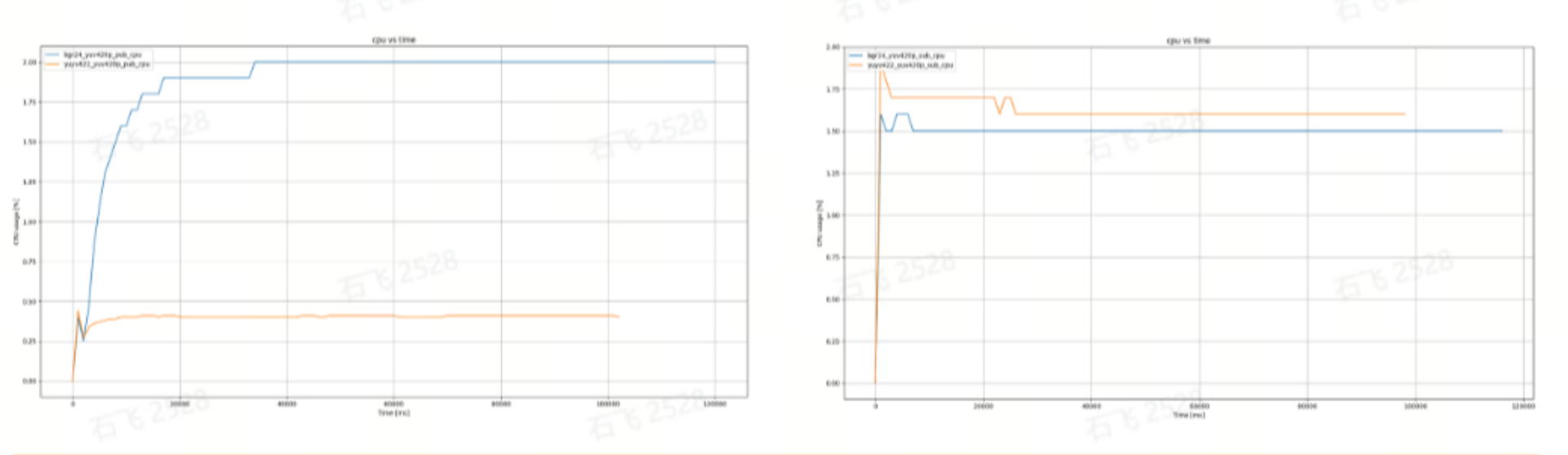
#### 4.1.1 Realsense D455

Realsense支持YUYV( `YUV422_YUY2` )格式的硬件输出，下图是输出YUYV时，对比优化前后的效果（左：发布端；右：订阅端）：

第一次测试：

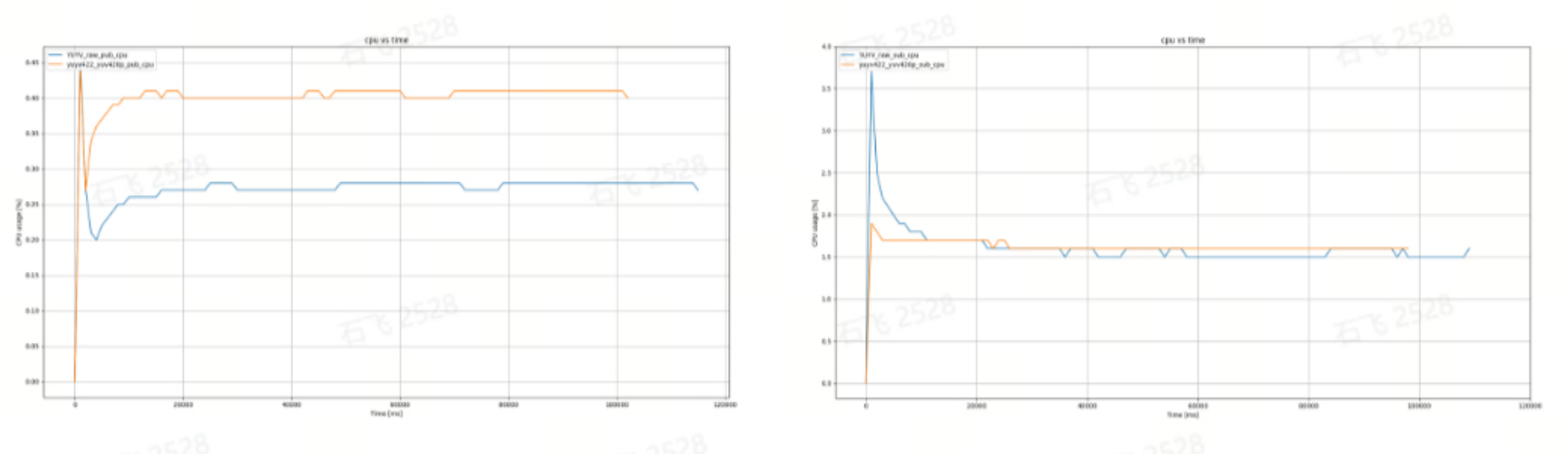


第二次测试：



> 发布端CPU占用率下降到原来的20%左右，订阅端略有提高，从方案上看我们的优化主要是Encoder前的数据处理，Encoder里用的格式都是 `AV_PIX_FMT_YUV420P`，不应该影响订阅端，需进一步测试。

Image Transport是支持订阅Raw Image的，如果不压缩，直接发布 `YUV422_YUY2` 格式的数据，在订阅端通过 `cv_bridge` 转马 `BGR8`，CPU占用率如下：

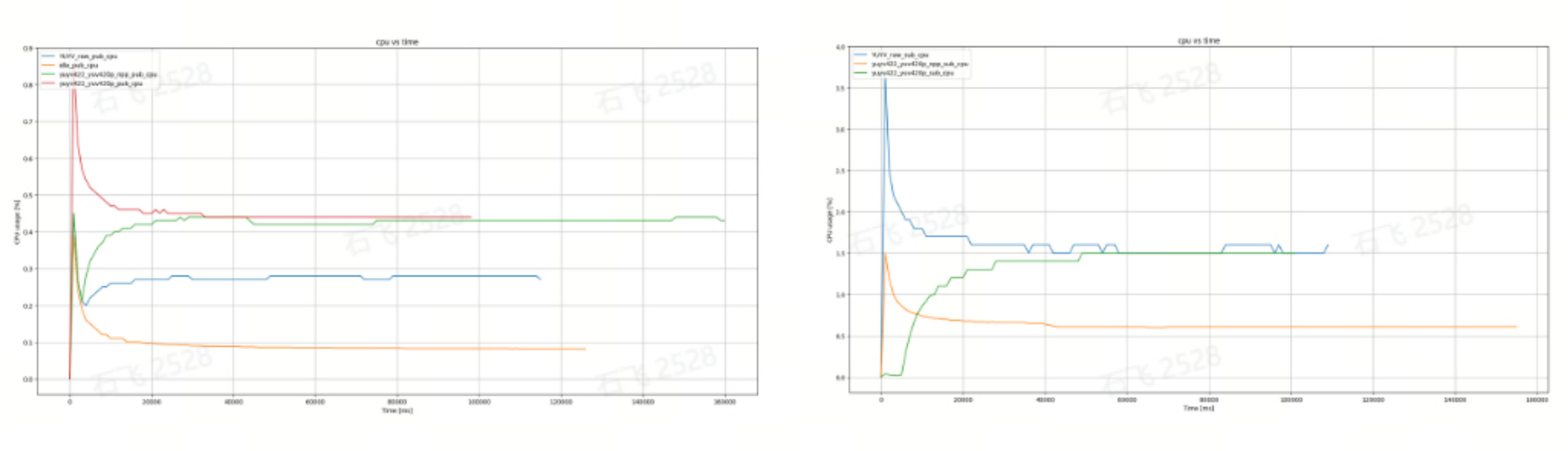


> 从上述对比数据看，直接传输YUV的开销最小，需进一步对于延时与频率稳定性进行进一步量化分析。

**2025年4月1日-优化了订阅端**

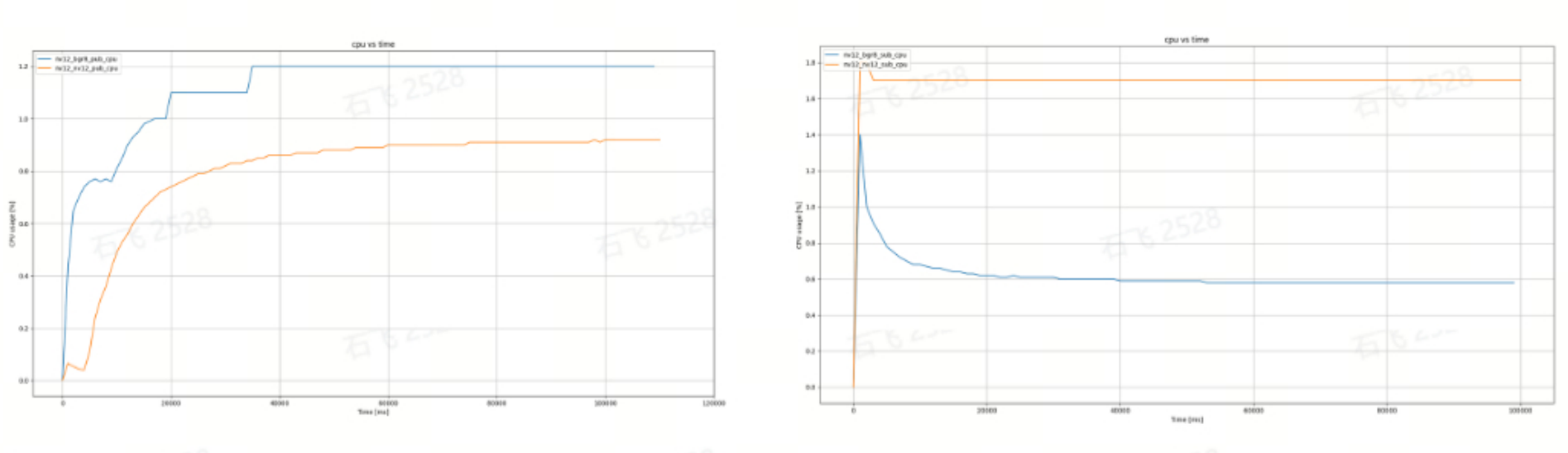
从左边的发布端看，H264压缩方式要比直接发布YUV的开销大，因为有 `YUV422_YUY2` 转 `YUV420P` 的环节，这一步仍然是由 `sws_scale` 在CPU上实现的。

从右边的订阅端看，直接发布 `YUV422_YUY2` 的Raw Data与H264解码优化之前的开销基本相同，因为都涉及YUV转BGR8，对H264解码优化后，可以看出CPU占用率大幅下降，大约只有原来的50%。



#### 4.1.2 肇观相机

原来的Image Transport插件不支持NV12格式，所以这里我们用肇观发布BGR8 Raw Image和H.264压缩格式进行对比左：发布端；右：订阅端）：

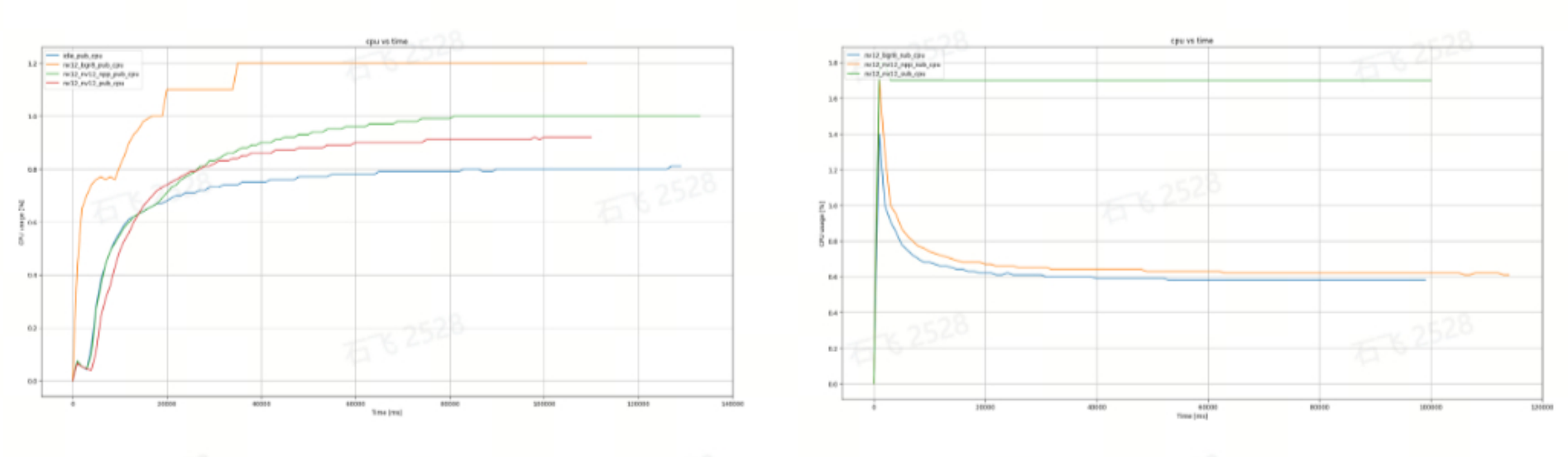


> 从当前数据看，在订阅端解码的开销太大，需进一步验证发布端直接发YUV422时的开销与延时情况。

**2025年4月1日-优化了订阅端**

这里对比的是发布端将 `NV12` 转成 `BGR8` 直接发布，和经过H264夺缩后发布，以及没有订阅IDLE状态下的CPU占用情况，可以看出通过H264压缩将相机USB传输（IDLE）之外的开销降了50%左右。

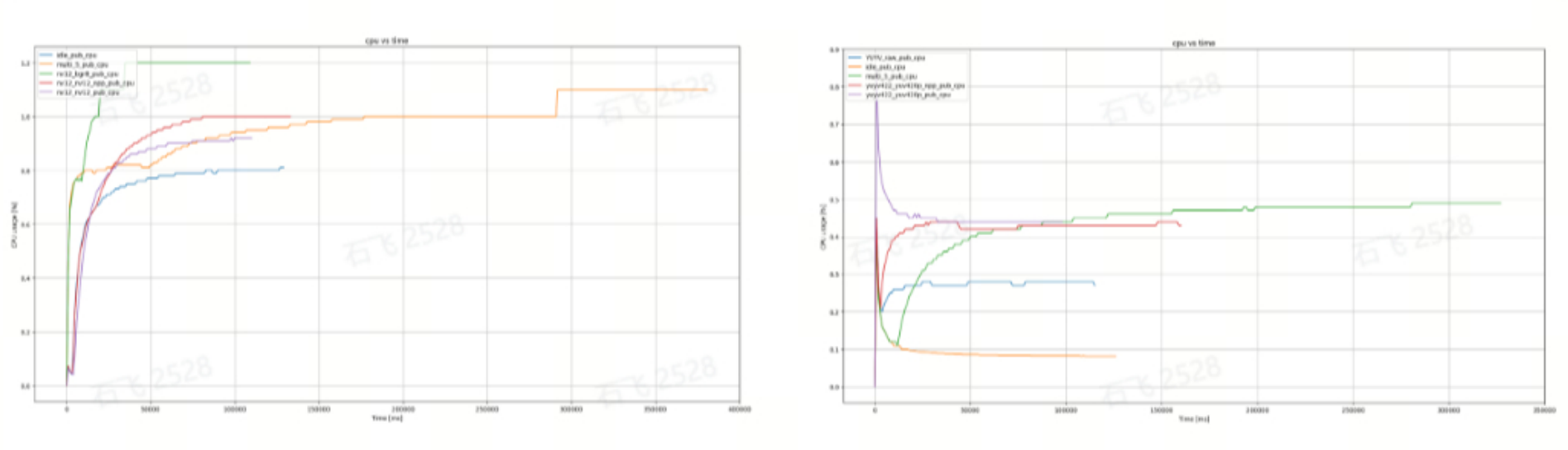
对比订阅端，优化后的开销只有原来的1/3左右，且与直接接收 `BGR8`（无色彩转换）的开销非常接近。



#### 4.1.3 多个订阅者

对于同一个Topic，每分钟增加一个订阅者，来查看发布端的CPU变化。左侧为肇观的数据，右侧为Realsense的数据。

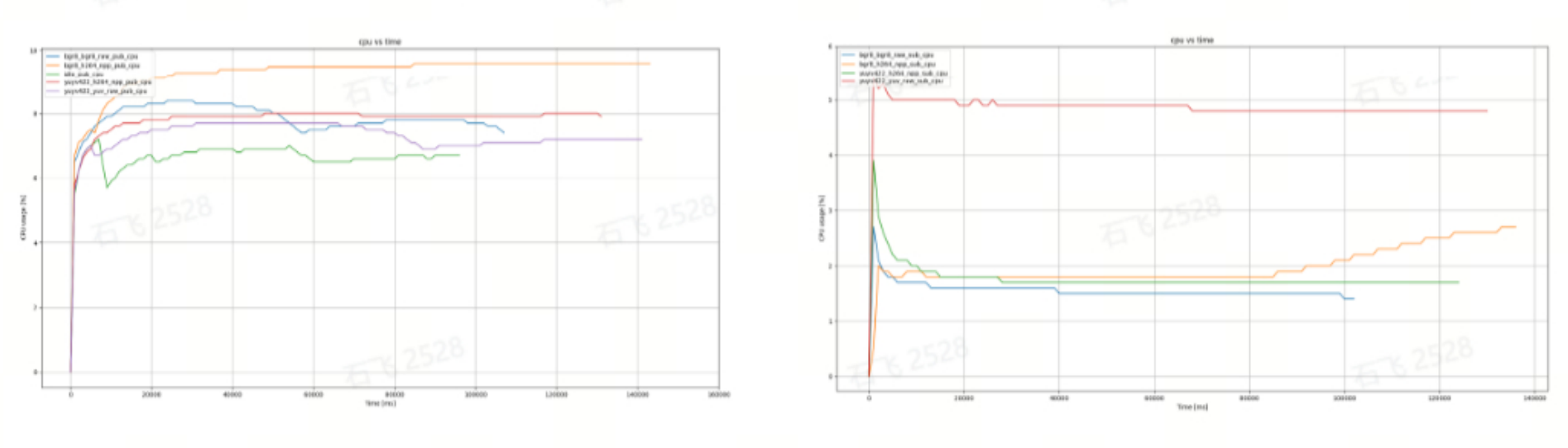
> 发布端的CPU占用率会随着订阅者的数据增加而变大，但增幅不大。以肇观为例，5个订阅者的CPU占用率不高于一路BGR8时的CPU占用率。



### 4.2 真机上的测试数据

#### 4.2.1 Realsense D435

Realsense Raw Data Format选择 `YUV422_YUY2`，H.264压缩传输的CPU开销是综合最优的。



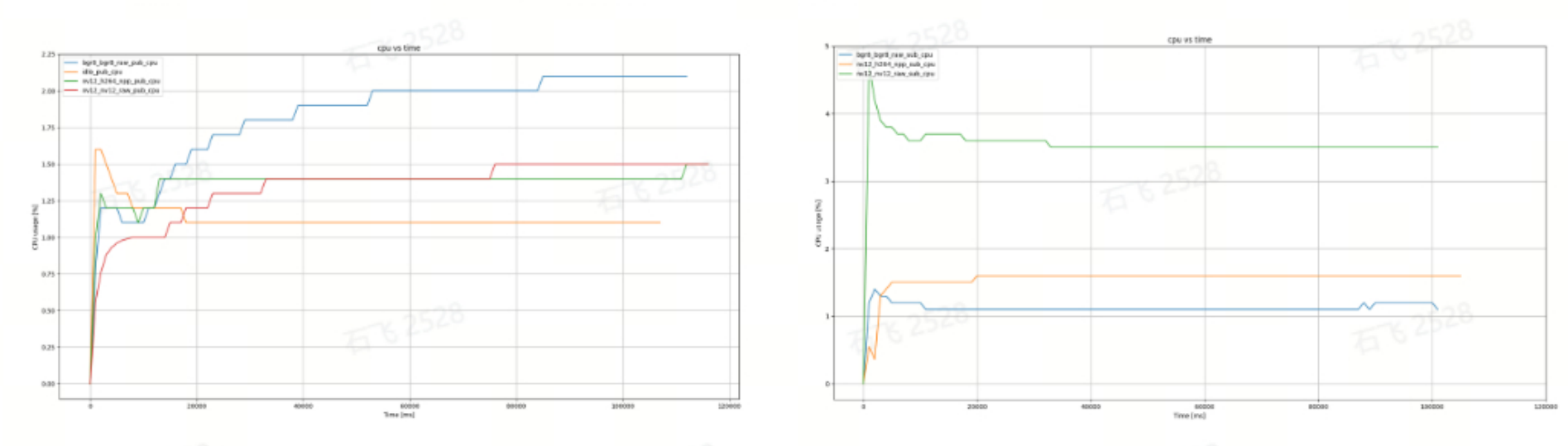
| Raw Data Format | Transport Type | Publisher CPU usage percentage | Subscriber CPU usage percentage | Total |
|---|---|---|---|---|
| `BGR8` | raw | 7.8 %<br>— bgr8_bgr8_raw_pub_cpu | 1.5 %<br>— bgr8_bgr8_raw_sub_cpu | 9.3 % |
| `BGR8` | foxglove | 9.5 %<br>— bgr8_h264_npp_pub_cpu | 1.8 %<br>— bgr8_h264_npp_sub_cpu | 11.3 % |
| `YUV422_YUY2` | foxglove | 8 %<br>— yuyv422_h264_npp_pub_cpu | 1.7 %<br>— yuyv422_h264_npp_sub_cpu | 9.7 % |
| `YUV422_YUY2` | raw | 7.8 %<br>— yuyv422_yuv_raw_pub_cpu | 4.8 %<br>— yuyv422_yuv_raw_sub_cpu | 12.6 % |
| **IDLE** | | 6.4 %<br>— idle_pub_cpu | | |

备注：当CPU开销接近时，压缩传输在帧率稳定性和延时上优势明显。

> 图像进程间传输Benchmark(Raw Data: BGR8)

#### 4.2.2 肇观相机

肇观相机Raw Data Format选择 `NV12`，H.264压缩传输的CPU开销是综合最优的。

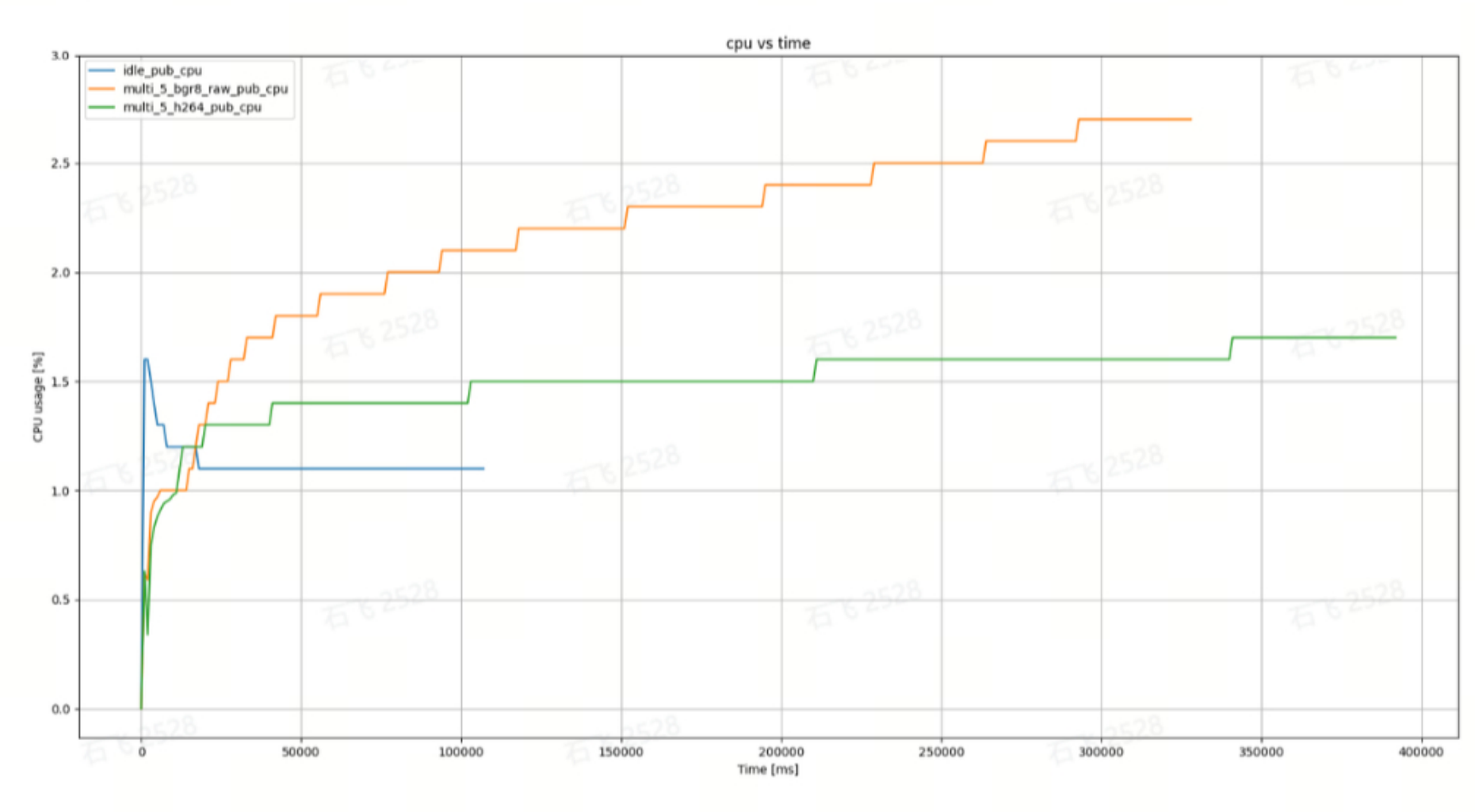


| Raw Data Format | Transport Type | Publisher CPU usage percentage | Subscriber CPU usage percentage | Total |
|---|---|---|---|---|
| `NV12` -> `BGR8` | raw | 2.0 %<br>— bgr8_bgr8_raw_pub_cpu | 1.2%<br>— bgr8_bgr8_raw_sub_cpu | 3.2 % |
| `NV12` | foxglove | 1.37 %<br>— nv12_h264_npp_pub_cpu | 1.5 %<br>— nv12_h264_npp_sub_cpu | 2.87 % |
| `NV12` | raw | 1.37 %<br>— nv12_nv12_raw_pub_cpu | 3.5 %<br>— nv12_nv12_raw_sub_cpu | 4.87 % |
| **IDLE** | | 1.1 %<br>— idle_pub_cpu | | |

#### 4.2.3 多个订阅者

对于同一个Topic，每分钟增加一个订阅者，来查看发布端的CPU变化。左侧为肇观的数据，右侧为Realsense的数据。

> 这里对BGR8和H264两种格式，进行了对比，H264的涨幅明显要小于BGR8的涨幅。



参考：

Convert NV12 to BGR by NVIDIA Performance Primitives

`frame_dec` 通过ffmpeg cuda解码器解码

```
    NppiSize oSizeROI;
    oSizeROI.width = frame_dec->width;
    oSizeROI.height = frame_dec->height;
    DBUG_PRINT("width: %d, height: %d, stepBytes: %d\n", oSizeROI.width, oSizeROI.height, ste
    // NppStatus stat = nppiNV12ToBGR_8u_P2C3R(frame_dec->data, frame_dec->width, bgrData, fr
    NppStatus stat;#ifdef USE_709
    stat = nppiNV12ToBGR_8u_P2C3R(frame_dec->data, frame_dec->linesize[0], bgrData, frame_dec
#else
    stat = nppiNV12ToBGR_709HDTV_8u_P2C3R(frame_dec->data, frame_dec->linesize[0], bgrData, f
#endifunsigned char *data = (unsigned char *)malloc(frame_dec->width * frame_dec->height * 3)
    cudaMemcpy(data, bgrData, frame_dec->height * frame_dec->width * 3, cudaMemcpyDeviceToHos
    cv::Mat mat_test(frame_dec->height, frame_dec->width, CV_8UC3, data);
    imwrite("test.jpg", mat_test);
    free(data);
    nppiFree(bgrData);exit(0);
```

反向引用 (1)　本文引用 (1)　关系图
2025系统架构周报

真诚点赞，手留余香
输入评论